

10

Causal Inference and Machine Learning – Advanced Estimators, Experiments, Evaluations, and More

Welcome to *Chapter 10*!

We closed the previous chapter by discussing meta-learners. We started with a single model S-Learner and finished with a complex X-Learner that required us to train five machine learning models behind the scenes!

Each new model was an attempt to overcome the limitations of its predecessors. In this chapter, we'll continue to walk the path of improvement. Moreover, we'll integrate some of the approaches introduced in the previous chapter in order to make our estimates better and decrease their variance.

In this chapter, we'll learn about **doubly robust (DR)** methods, **double machine learning (DML)**, and Causal Forests. By the end of this chapter, you'll have learned how these methods work and how to implement them using EconML by applying them to real-world experimental data. You'll also have learned about the concept of counterfactual explanations.

In this chapter, we'll cover the following topics:

- Doubly robust methods
- Double machine learning
- Targeted maximum likelihood estimator
- Causal Forests
- Heterogeneous treatment effects with experimental data
- Counterfactual explanations

Doubly robust methods – let’s get more!

So far, we’ve discussed a broad range of methods that can be used to estimate causal effects. In earlier chapters, we discussed linear regression; in later chapters, we discussed matching, propensity score weighting, and the meta-learner framework. The latter allowed us to go beyond the limitations of linear regression by plugging arbitrary machine learning models as base learners. Meta-learners turned out to be very flexible as they offer all the benefits of contemporary machine learning.

Do we need another thing?

As we learned, propensity scores alone can be used to deconfound the data (as in propensity score weighting; note that this only holds for observed confounding). The same is true for regression models, where we can deconfound the data by simply controlling for the right variable(s) by including them in the regression formula. Propensity score models are sometimes referred to as **treatment models** (as they aim to predict the treatment from confounders) and regression models are referred to as **outcome models** (as they aim to predict the outcome from the treatment and other relevant variables; note that we use the term *regression models* in a broad sense, which includes but is not limited to linear regression).

Isn’t the fact that both *treatment* and *outcome* models can be used to achieve the same goal interesting?

Let’s see.

A graphical approach is usually helpful when we want to examine questions like this. *Figure 10.1* presents two **directed acyclic graphs (DAGs)**:

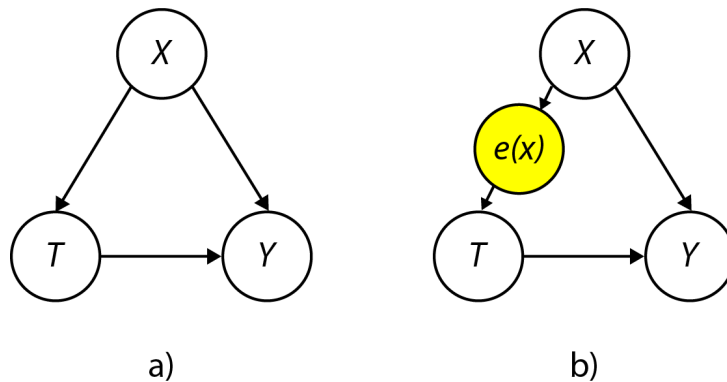


Figure 10.1 – Two models – without and with a propensity score

In panel **a**), we see a model with a treatment, T , an outcome, Y , and a confounder, X . In panel **b**), we see the same model, but with the propensity score, $e(X)$, added on the path between X and T . As a refresher, the propensity score is defined as follows:

$$e(X) = P(T|X)$$

Adding a new node representing the propensity score to the graph is possible thanks to the **propensity score theorem** that we learned about in the previous chapter.

The theorem states that if we have unconfoundedness given X , we will also have unconfoundedness given the propensity score of X , assuming positivity (Rosenbaum & Rubin, 1983). Thanks to this, we can use the propensity score as a full mediator of the relationship between X and T . Note that this gives us a possibility to close the backdoor path between T and Y in three ways (Figure 10.1; panel *b*):

- By controlling for X
- By controlling for $e(X)$
- By controlling for both of them

Each of these ways will deconfound the relationship between the treatment and the outcome.

Interestingly, although all three ways to deconfound the data are equivalent from a graphical point of view, they might lead to different estimation errors. In particular, although the *estimands* obtained using X and $e(X)$ are equivalent ($P(Y|T, X) = P(Y|T, e(X))$), the *estimators* estimating these quantities might have different errors. In particular, in certain cases, one of the models might be correct with a near-zero error, while the other might be misspecified with arbitrary bias.

What if we could combine both estimators in a way that would allow us to automatically choose the better model?

It turns out that this is exactly what DR methods do. We get a model that automatically *switches* between the outcome model and the treatment model. The bias of DR estimators scales as the *product* of the errors of the *treatment* and *outcome* models (Hernán & Robins, 2020; p. 229).

Let's reiterate this – the DR estimator will give asymptotically unbiased estimates whenever one of the models (outcome or treatment) is correctly specified. This is a really powerful property!

Moreover, DR methods can be seen as a subset of the meta-learner framework, meaning that we can use arbitrary estimators in order to compute the outcome and treatment models. DR estimators are guaranteed to be consistent when either of the models (*treatment* or *outcome*) is correct, but not necessarily both models. DR is a large-sample property, which means that if our sample size is small, we might not get the benefits.

Doubly robust is not equal to bulletproof...

Although theoretically speaking, the treatment and outcome models are equivalent in the DR estimator, some practitioners and researchers report that misspecification of the outcome model leads to a significant bias *even if the treatment model is correct* (Li, 2021).

Moreover, when both models (treatment and outcome) are even moderately misspecified, DR estimators can have high bias and high variance (Kang & Schafer, 2007).

...but it can bring a lot of value

If both models – outcome and treatment – are correct, the DR estimator will have a smaller variance than **inverse probability weighting (IPW)**, at least in large samples (Li, 2021).

Speaking from experience, it will usually also have a smaller variance than the meta-learners introduced in the previous chapter. This observation is supported by theory. The DR estimator can be decomposed as an S-Learner with an additional adjustment term (for details, see Courthoud, 2022).

The secret doubly robust sauce

Although many advances in DR estimators are fairly recent (see Tan et al., 2022), the method's origins date back to the 1994 paper *Estimation of Regression Coefficients When Some Regressors are not Always Observed* by Robins et al. (1994). Interestingly, the authors originally considered the method in the context of the missing data problem.

To be fair, the basic idea of double robustness dates back even earlier – to Cassel et al.'s 1976 paper. Cassel and colleagues (1976) proposed an estimator virtually identical to what we call the DR estimator today, with the difference that they only considered a case with known (as opposed to estimated) propensity scores.

Causal inference and missing data

The fundamental problem of causal inference (Holland, 1986) that we introduced in *Chapter 2* is that we can never observe all potential outcomes at the same time. One view on causal inference that stems from this perspective is that it is a missing data problem. This idea inspired researchers and practitioners to use causal methodologies for data imputation purposes (e.g., Mohan & Pearl, 2021) and vice versa. For an overview, check out Ding & Li (2018).

We're now ready to define a basic DR estimator formally (Facure, 2020):

$$\hat{\tau}_{ATE} = \frac{1}{N} \sum_i \left(\frac{T_i(Y_i - \mu_1(X_i))}{\hat{e}(X_i)} + \mu_1(X_i) \right) - \frac{1}{N} \sum_i \left(\frac{(1 - T_i)(Y_i - \mu_0(X_i))}{1 - \hat{e}(X_i)} + \mu_0(X_i) \right)$$

In the preceding formula, T_i is the treatment for unit i , $\hat{e}(X_i)$ is the estimate of the propensity score, μ_0 and μ_1 are models that estimate $E[Y|X, T = t]$ with $t = 0$ and $t = 1$, respectively, and Y is the outcome variable. Note that we define an estimator for the **average treatment effect (ATE)** here, but DR methods can be applied to estimate **conditional average treatment effects (CATE)** as well.

Let's unpack the formula.

We'll set the stage in the following way:

1. Divide the big formula into two chunks (on the left of the main minus sign and on the right).
2. Focus on the part inside the brackets on the left.
3. Develop the intuition for this part.
4. Generalize this intuition to the entire formula.

We've supplied a color-coded formula in *Figure 10.2* to make the process easier to understand.

$$\hat{\tau}_{ATE} = \frac{1}{N} \sum_i \left(\underbrace{\frac{T_i(Y_i - \mu_1(X_i))}{\hat{e}(X_i)}}_{\mu_1 = \text{model with treatment}} + \mu_1(X_i) \right) - \frac{1}{N} \sum_i \left(\underbrace{\frac{(1 - T_i)(Y_i - \mu_0(X_i))}{1 - \hat{e}(X_i)}}_{\mu_0 = \text{model without treatment}} + \mu_0(X_i) \right)$$

Figure 10.2 – A color-coded formula for the DR estimator

The two main parts of the formula in *Figure 10.2* represent models under treatment (left; orange) and under no treatment (right; green). The purple parts represent averaging.

Let's focus on the orange part first.

Let's say that our model, μ_1 , returns a perfect prediction for observation X_i , but our propensity estimate, $\hat{e}(X_i)$, is way off. In such a case, this prediction will be equal to Y_i . Therefore, $Y_i - \mu_1(X_i)$ will be 0. This will make the whole fraction expression $\left(\frac{T_i(Y_i - \mu_1(X_i))}{\hat{e}(X_i)}\right)$ in the orange part in *Figure 10.2* equal to 0, leaving us with $\mu_1(X_i)$ after the plus sign. Because μ_1 returned a perfect prediction, $\mu_1(X_i)$ is a perfect estimate and we're happy to use it.

Note that the same logic holds for the green part of the formula in *Figure 10.2*.

We can also show that in the case of a misspecified outcome model, the opposite will happen – the outcome model will be weighted down to 0 and the treatment model will take the lead in the estimate. Showing this mechanism requires some algebraic transformations, which we will skip here. If you're interested in more details, Chapter 12 of Matheus Facure's online book contains an accessible explanation: <https://bit.ly/DoublyRobust>.

Doubly robust estimator versus assumptions

As with all the methods discussed in the previous chapter, the DR estimator works properly when there's no hidden confounding (i.e., X contains all the variables we need to control for and no variables that introduce bias). For precise estimates of CATE, we also need to include relevant effect modifiers in X .

Additionally, note that *positivity assumption* violations might lead to high variance for DR estimators (Li, 2020).

DR-Learner – crossing the chasm

DR-Learner (*DR* comes from *doubly robust*) is a DR meta-learner model. EconML offers a couple of variants of DR-Learner. The simplest one is the linear variant implemented in the `LinearDRLearner` class. This model is in fact more complex than the DR estimator we described earlier.

It uses **cross-fitting** and **orthogonalization** (we'll discuss both terms in the next section) and an additional model to estimate CATE directly. For `LinearDRLearner`, this additional model defaults to linear regression, but this can be changed to an arbitrary model in another class called `DRLearner`.

The code for the following experiments can be found in the notebook for *Chapter 10* (<https://bit.ly/causal-ntbk-10>).

We will continue with the same data that we used for meta-learners in the previous chapter and so we'll skip the graph creation and estimand identification steps here. You can find the full pipeline implemented in the notebook accompanying this chapter.

As in the case of meta-learners, we'll use DoWhy's `CausalModel` API to call EconML's estimators.

Let's compute our estimate using `LinearDRLearner`:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dr.LinearDRLearner',
    target_units='ate',
    method_params={
        'init_params': {
            # Specify treatment and outcome models
            'model_propensity': LogisticRegression(),
            'model_regression': LGBMRegressor(n_estimators=
```

```

        1000, max_depth=10)
    },
    'fit_params': {}
})

```

First, we use `'backdoor.econml.dr.LinearDRLearner'` as a method name. We specify the ATE as a target unit (but CATE estimates will still be available to us) and pass model-specific parameters as a dictionary. Here, we only use two parameters to specify the outcome model (`'model_regression'`) and the treatment model (`'model_propensity'`). We picked simple logistic regression for the latter and an LGBM regressor for the former.

As a reminder, logistic regression is often the go-to method for propensity score modeling as its probability output is well calibrated. Its limitation is that it cannot model interactions directly, so if you expect that interaction terms might be important for your propensity model, you might want to specify interaction terms explicitly in your dataset (you can do so by multiplying the features that interact with each other and adding the result as a new feature(s) to your dataset) or go for another model.

In the latter case, remember to scale the probability output.

Let's generate predictions:

```

effect_pred = model.causal_estimator.effect(
    earnings_interaction_test.drop(['true_effect',
    'took_a_course'], axis=1))

effect_true = earnings_interaction_test['true_effect']
               ].values

```

Now, we're ready to evaluate our model. Let's check the value of the **mean absolute percentage error (MAPE)**:

```

mean_absolute_percentage_error(effect_true, effect_pred)

```

This gives us the following result:

```

0.00623739241153059

```

This is *six times* lower than X-Learner's error in the previous chapter (3.6%)!

Let's plot the predictions versus the true effect (*Figure 10.3*):

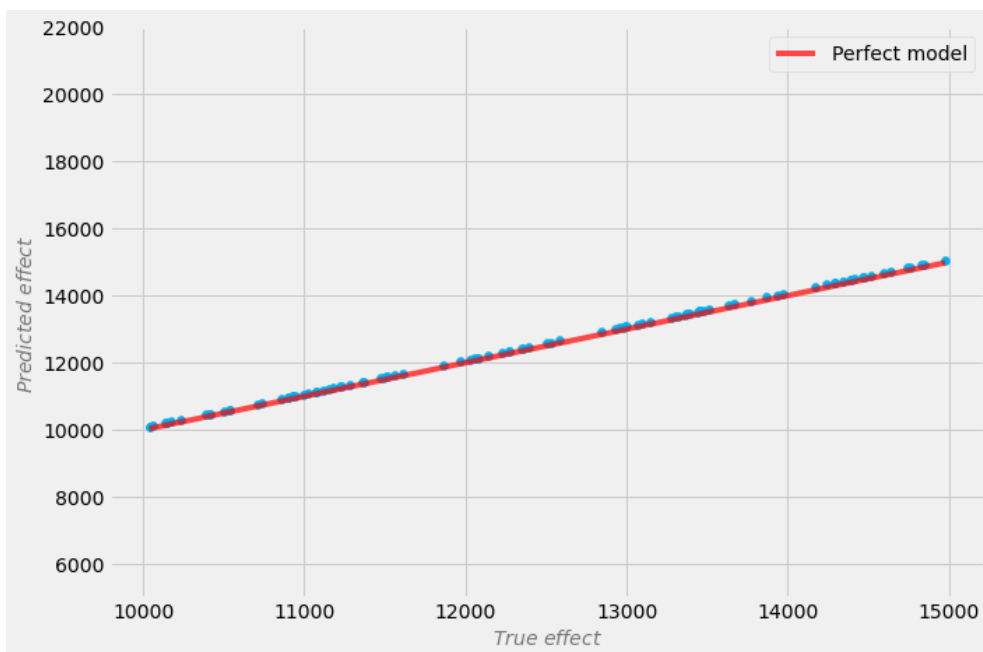


Figure 10.3 – True versus predicted effect for the linear DR-Learner model

This looks pretty impressive!

Let's think about what could help the model get such a good fit.

As we said earlier, DR methods can be a benefit when both models – outcome and treatment – are well specified. This is the case in our example. This allows the model to achieve a small variance compared to the meta-learners in the previous chapter.

The bias is also very small, although you can see in the plot that the model slightly, but systematically, overestimates the true effect.

Note that the scale of the plot plays a role in forming our impressions regarding the model's performance. For ease of comparison, we kept the ranges for both axes identical to the ones in the previous chapter. To an extent, this can mask the model's errors visually, but it does not change the fact that the improvements over meta-learners from *Chapter 9* are significant.

Let's address one more question that some of us might have in mind – we used a model called `LinearDRLearner`. Why do we call it linear if we used a non-linear boosting model as an outcome model? The answer is that the orthogonalization procedure that we'll describe in the next section allows us to model non-linearities in the data while preserving linearity in causal parameters.

This setting is a good-enough fit for a broad category of problems, yet if you need more flexibility, you can use the `DRLearner` class and choose an arbitrary non-linear final model.

Note that using super-powerful models might sometimes be overkill. To illustrate this, let's use `DRLearner` with a relatively deep boosting regressor as a final model and apply it to our data:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dr.DRLearner',
    target_units='ate',
    method_params={
        'init_params': {
            'model_propensity': LogisticRegression(),
            'model_regression': LGBMRegressor(
                n_estimators=1000, max_depth=10),
            'model_final': LGBMRegressor(
                n_estimators=500, max_depth=10),
        },
        'fit_params': {}
    })
```

This model's MAPE is over 7.6% – over 10 times higher than for a simple linear DR-Learner. *Figure 10.4* presents the results:

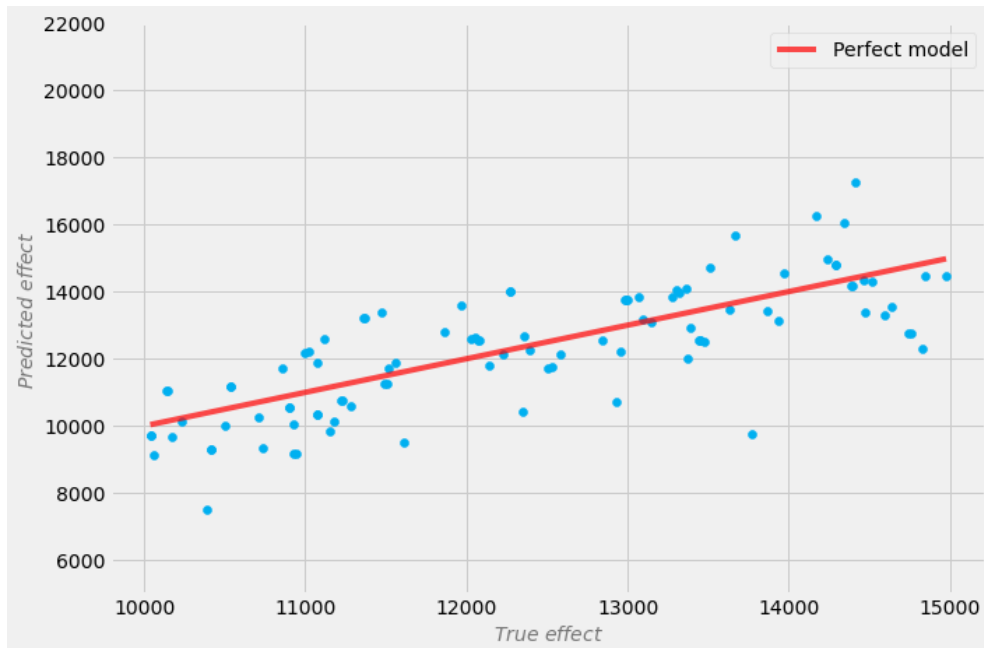


Figure 10.4 – True versus predicted effect for the non-linear DR-Learner model

As we can see, the non-linear model has a much higher variance on the test set. Try to decrease the number of estimators and the maximum depth in the final model and see how it affects the error.

Spoiler alert – you should expect some improvements!

DR-Learners – more options

EconML provides us with a total of four different DR-Learners. On top of the two that we’ve discussed before, we also have `SparseLinearDRLearner`, which uses debiased lasso regression (Bühlmann & van de Geer, 2011; van de Geer et al., 2013) as a final stage algorithm, and `ForestDRLearner`, which uses *Causal Forest* as the final stage algorithm.

We’ll now discuss them briefly.

`SparseLinearDRLearner` uses an L1-penalized regression, which means that the algorithm can effectively perform automatic feature selection by weighting the coefficients for some features down to 0. A sparse linear DR-Learner will be a good choice whenever the number of features is large.

It shines in particular when the number of features is close to or larger than the number of observations, where a regular linear DR-Learner would fail. The model will also provide you with valid confidence intervals. Its main limitation is that – similar to `LinearDRLearner` – it expects that the treatment effect is linear.

`ForestDRLearner` uses the Causal Forest algorithm (Wager & Athey, 2018) as the final model. We’ll discuss Causal Forests in greater detail later in this chapter. Forest DR-Learner is great at handling multidimensional data, and unlike `SparseLinearDRLearner` is not limited to linear cases.

If obtaining confidence intervals is important to you, `ForestDRLearner` can provide them through bootstrapping. Note that this will significantly impact computation times for the model, in particular in large samples.

We will discuss how to obtain confidence intervals for estimators that do not support such an option natively later in the chapter.

Now, let’s see another interesting DR estimator that can bring advantages under (limited) positivity violations (Porter et al., 2011) or smaller sample sizes.

Targeted maximum likelihood estimator

DR estimators come in various shapes and tastes. One popular DR method is called TMLE. It was originally introduced by van der Laan and Rubin in their paper *Targeted Maximum Likelihood Learning* (van der Laan & Rubin, 2006).

TMLE is a semi-parametric method and allows us to use machine learning algorithms while yielding asymptotic properties for statistical inference. In other words, we can use complex machine learning models but retain some of the convenience of traditional statistical methods, including generating

valid confidence intervals without bootstrapping. Moreover, TMLE uses an extra *targeting* step to optimize the bias-variance trade-off when estimating the causal effect.

Note that some of TMLE’s properties are similar to these of DR-Learner or DML (which we will discuss in the next section). A detailed comparison between the three is beyond the scope of this book, but you’ll find links to the resources containing detailed TMLE and DML comparisons at the end of the next section.

In this section, we’ll explain the mechanics behind TMLE, so you can understand the main factors contributing to its uniqueness.

Note that the TMLE estimator we introduce in this chapter works for binary treatments and binary outcomes only. The method can be extended to continuous outcomes under certain conditions (Gruber & van der Laan, 2010).

As TMLE isn’t currently available in EconML, we won’t implement it here. Instead, we’ll look into the algorithm mechanics step by step and share the resources demonstrating how to use it in other Python packages, such as IBM’s `causalml` or `zEpid`.

TMLE can be used with any machine learning algorithm and in this sense can be treated as a *meta-learner*.

There are eight steps to implement TMLE:

1. Train a model, μ , that models $Y \sim X + T$, where Y is a binary outcome, X is a feature matrix, and T is a binary treatment. Arbitrary machine learning models compatible with the outcome and features can be used here.
2. Generate the following predictions (note that all $\hat{y}$ s are probabilities, not labels):

- A. For each observation, predict the outcome for the actual treatment value:

$$\hat{y}_t = \mu(X, T = t)$$

- B. For each observation, predict the outcome under no treatment:

$$\hat{y}_0 = \mu(X, T = 0)$$

- C. For each observation, predict the outcome under treatment:

$$\hat{y}_1 = \mu(X, T = 1)$$

3. Estimate the propensity score for each observation: $\hat{e}(X) = P(T = 1|X)$.

4. Compute the following quantity:

$$H(T, X) = \frac{1_{\{T=1\}}}{\hat{e}(X)} - \frac{1_{\{T=0\}}}{1 - \hat{e}(X)}$$

where $1_{(T=t)}$ is an indicator function (refer to *Chapter 2* (the *Associations* section) for a definition). The quantity, $H(T, X)$, is known in the literature as the **clever covariate** and it will help us update our estimates to reduce bias.

5. Estimate the **fluctuation parameter**, which will help us calculate the variance necessary to obtain confidence intervals for our estimates. This can be achieved by doing the following:

- A. Fitting a logistic regression model:

$$\gamma(T, X, \hat{y}_t) = Y \sim [-1 + \text{logit}(\hat{y}_t)] + H(T, X)$$

In the preceding formula, $[-1 + \text{logit}(\hat{y}_t)]$ means *do not fit intercept* (-1); instead, use a fixed vector $\text{logit}(\hat{y}_t)$.

By saying *fixed vector*, we mean that we take a *whole vector of values* (in this case, a vector of predictions coming from the model, $\mu(X, T = t)$), and use this vector as an intercept, instead of using just a single number (scalar). Specifically, the intercept value for each row in our dataset is the prediction of the $\mu(X, T = t)$ model for that particular row.

- B. Obtaining the coefficient for $H(T, X)$ from the preceding logistic regression model. We call this coefficient the *fluctuation parameter* and use $\hat{e}$ to denote it.

6. Update the predictions from *step 2* using the fluctuation parameter, $\hat{e}$. We'll use asterisk notation for the updated predictions:

$$A. \hat{y}_t^* = \text{expit}(\text{logit}(\hat{y}_t) + \hat{e}H(T = t, X))$$

$$B. \hat{y}_0^* = \text{expit}(\text{logit}(\hat{y}_0) + \hat{e}H(T = 0, X))$$

$$C. \hat{y}_1^* = \text{expit}(\text{logit}(\hat{y}_1) + \hat{e}H(T = 1, X))$$

The *expit* function is the inverse of the *logit* function. We need it here to go back from *logits* to *probabilities*. To learn more about *logit* and *expit*, check out this great blog post by Kenneth Tay: <https://bit.ly/LogitExpit>.

7. Compute the estimate of the ATE (this can be also adapted for CATE):

$$\hat{\tau}_{ATE} = \frac{1}{N} \sum_i \hat{y}_1^* - \hat{y}_0^*$$

8. Compute the confidence intervals:

- A. Calculate the values of the **influence function** ($\widehat{IF}$):

$$\widehat{IF} = (Y - \hat{y}_t^*)H(T, X) + \hat{y}_1^* - \hat{y}_0^* - \hat{\tau}_{ATE}$$

- B. Compute the estimate of the standard error:

$$\widehat{SE} = \sqrt{\frac{\text{var}(\widehat{IF})}{N}}$$

A. Get the 95% intervals:

$$CI_{lower} = \hat{\tau}_{ATE} - 1.96 \widehat{SE}$$

$$CI_{upper} = \hat{\tau}_{ATE} + 1.96 \widehat{SE}$$

That's the mechanics of TMLE.

Following the formulas, you can relatively easily implement it from scratch yourself in Python, but you don't have to!

TMLE has already been implemented in IBM's *causallib* library by Ehud Karavani, as well as in *zEpid* – an epidemiology analysis library created by computational epidemiologist Paul Zivich.

You can find *causallib*'s documentation for TMLE here: <https://bit.ly/TMLEDocs>. An example notebook can be found here: <https://bit.ly/CausallibTMLEExample>. The documentation for *zEpid*'s implementation of TMLE can be found here: <https://bit.ly/zEpidTMLEDocs>.

For an excellent visual introduction to TMLE, check out the series of blog posts by Katherine Hoffman: <https://bit.ly/KatHoffmanTMLE>. Her great work helped me structure this section more clearly and I hope this will also translate into more clarity for you.

It's time to conclude this section.

We learned that DR methods rely on two models: the *outcome model* and the *treatment model* (also called the *propensity model*). Both models can be estimated using any compatible machine learning technique – whether linear models, tree-based methods, or neural networks.

The core strength of DR models is that even if only one model (e.g., the outcome model) is specified correctly, they will still give consistent estimates. When both models (outcome and treatment) are specified correctly, DR estimators tend to have very low variance, which makes them even more attractive.

TMLE – an extension of the classic DR estimator – offers us the advantages of parametric models and good statistical properties while allowing us to use flexible machine learning models as base learners. Moreover, it can outperform other DR methods under positivity violations and can be an advantage for smaller sample sizes. Perhaps that's one of the reasons TMLE has gained significant traction lately, even though the original idea dates back to 2006.

In the next section, we'll dive into another exciting area of causal inference, called *double machine learning*, which offers some improvements over the DR framework.

If machine learning is cool, how about double machine learning?

The title of this section is inspired by Brady Neal's video (<https://bit.ly/BradyNealDML>) where he (jokingly) suggests that DML “*is maybe twice as cool*” as regular machine learning. Let's see!

DML, also known as **debiased machine learning** or **orthogonal machine learning**, is another causal framework with the root *double* in its name. In DML – similarly to DR methods – we fit two models that estimate different parts of the relationships in the data.

DML can be implemented using arbitrary base estimators, and in this sense, it also belongs to the meta-learner family. At the same time, unlike S-, T- and X-Learners, the framework comes with a strong theoretical background and unique architectural solutions.

In this section, we'll introduce the main concepts behind DML. After that, we'll apply it to our `earnings` dataset using DoWhy's API. We'll discuss some popular myths that have arisen around DML and explore the framework's main limitations. Finally, we'll compare it to DR estimators and present some practical guidelines for when to choose DML over DR-Learner.

Why DML and what's so double about it?

Similar to DR methods, DML also uses two estimators in order to model data, yet it does it slightly differently. Before we jump into the technicalities, let's discuss what the main motivations behind DML are.

DML was first proposed by MIT statistician and economist Victor Chernozhukov and colleagues (Chernozhukov et al., 2018). The authors' motivation was to build a causal estimator that can leverage the flexibility of non-parametric machine learning models while achieving low bias and offering *valid confidence intervals* (note how similar these ideas are to what we saw for TMLE).

In particular, the authors built DML in a way that makes it **root-n consistent**. We say that an estimator is consistent when its error goes down with the sample size. Intuitively, root-n consistency is an indicator that the estimation error goes to 0 at a rate of $\frac{1}{\sqrt{n}}$ when the sample size (n) goes to infinity.

More on consistency

Estimator **consistency** is important (not only in causality) because it tells us whether the estimator will converge to the true value with a large enough sample size. Formally, we say that a consistent estimator will *converge in probability* to the true value.

Although consistency is an asymptotic property, consistent estimators usually show good behavior in practice (with finite and realistic sample sizes).

Inconsistent estimators, on the other hand, have a non-zero probability of non-convergence even for infinitely large samples.

For a lightweight introduction to consistency, root-n consistency, and convergence in probability, check out Kenneth Tay's blog post: <https://bit.ly/RootNConsistency>.

OK, so we have a consistent estimator that gives us the flexibility of machine learning and valid confidence intervals. How do we achieve these properties?

It turns out that there are two main sources of bias that we need to address.

The first comes from a mechanism that we usually see as beneficial in machine learning: **regularization**. Regularization is a set of techniques that helps machine learning models avoid overfitting. It turns out that it can lead to bias when we apply machine learning techniques to treatment effect estimation, preventing us from achieving *root-n consistency*.

The second source of bias comes from overfitting. Let's start by addressing the latter.

Overfitting bias

DML solves the second problem using cross-fitting. The idea is as follows:

1. We split the data into two random partitions, D_0 and D_1 .
2. We fit the models (remember, we have two of them) on D_0 and estimate the quantity of interest on D_1 .
3. Next, we fit the models on D_1 and estimate on D_0 .
4. Finally, we average the two estimates and this gives us the final estimate.

This procedure might remind you of *cross-validation*. Cross-fitting and cross-validation both stem from the same idea of masking parts of the data, where masked partitions serve as a proxy for (future) unseen data. At first glance, cross-fitting and cross-validation have different goals (bias correction versus model evaluation), but the mechanics of both methods are virtually identical.

Most implementations of DML estimators perform cross-fitting on two partitions by default. From a theoretical point of view, the number of partitions (or *folds* as we call them in the context of *cross-validation*) has no impact on the estimator's performance (asymptotically). That said, *in practice*, DML tends to work better with four or five partitions than with just two, especially on smaller datasets. Although this is not a rule, these differences seem to have a *relatively systematic* character (Chernozhukov et al., 2018, p. C24).

Regularization bias

After crossing overfitting bias off our to-do list, we're now ready to tackle **regularization bias**.

DML addresses this challenge using a technique called **orthogonalization**. The procedure is inspired by the **Frisch-Waugh-Lovell (FWL)** theorem.

To get an understanding of FWL, let's start with the following linear regression equation:

$$Y = \beta_1 T + \beta_2 X + \epsilon$$

FWL states that we can obtain β_1 not only by fitting the model described by the preceding formula but also by carrying out the following steps:

1. Regress T on X using linear regression: $\hat{t} = \gamma X$.
2. Regress Y on X using linear regression: $\hat{y} = \psi X$.

3. Compute the residuals from regressions from points 1 and 2:

- A. $\delta_T = T - \hat{t}$

- B. $\delta_Y = Y - \hat{y}$

4. Regress residuals δ_Y on residuals δ_T using linear regression: $\hat{\delta}_T = \alpha_1 \delta_Y$.

It turns out that the coefficient, α_1 , from the last equation is *equal* to the coefficient, β_1 , from the original regression equation (note that if you code it yourself, you may get minor differences between β_1 and α_1 ; they might come from numerical imprecisions and/or the implementation details of the packages you use). For a proof of the FWL theorem, including R code examples, check out <https://bit.ly/FWLProof>.

Orthogonalization is a modification of the FWL procedure that allows us to use arbitrary machine learning models instead of linear regression in *steps 1* and *2* (note that the last model from *step 4* remains linear in the linear DML estimator (Chernozhukov et al., 2018); this limitation does not apply to non-parametric DML).

The core strengths of DML come from its ability to model complex non-linear relationships in the data. Let's re-write the original linear regression equation to allow for non-linearities. This will allow us to demonstrate the power of DML more clearly. We will start with a so-called *partially linear model*:

$$Y = \theta T + f_Y(X) + \epsilon_Y$$

$$T = f_T(X) + \epsilon_T$$

In the preceding formulas, Y is the outcome, T is the treatment, $f_Y(\cdot)$ and $f_T(\cdot)$ are arbitrary and possibly non-linear functions, X is a set of predictors that *includes the confounders*, and the ϵ s represent error terms. In this formulation, θ is the causal parameter that we want to estimate.

By applying orthogonalization, we can separate the estimation of the causal parameter, θ , from estimating the so-called **nuisance parameter** that models the impact of non-causal predictors in the model (the nuisance parameter in our partial linear model is represented by $f_Y(\cdot)$, which is a function; that's why the nuisance parameter is sometimes also referred to as the **nuisance function**).

The last step of the orthogonalization procedure (which is analogous to the last step of FWL) will provide us with an estimate of the θ parameter that is *free* of the *regularization bias*!

Moreover, this means that we can get the benefits of parametric statistical inference and don't care about the exact functional form of the nuisance parameter. This gives us great flexibility!

Before we conclude this subsection, let's summarize the key points. DML uses two machine learning models in order to estimate the causal effect. These models are sometimes called the *treatment* and the *outcome* models (as they were in the case of DR methods), but note that the treatment model in DML does not estimate the propensity score, but rather models treatment directly.

Additionally, we fit one more model to estimate the causal parameter, θ , from the residuals. This model is known as the *final model*. In linear DML, this model is – by design – a linear regression, but in generalized, non-parametric DML, an arbitrary machine learning regressor can be used.

DML is an estimator with low variance and bias and can provide us with valid confidence intervals. It also natively supports both discrete and continuous treatments.

DML with DoWhy and EconML

Let's implement a linear DML estimator and apply it to our data. We'll skip graph creation, model initialization, and estimand identification steps as they are identical to what we've done for previous models. Note that this modular logic offered by DoWhy makes experimentation with various methods seamless, as problem definition and estimand identification are completely independent of the estimation step.

To use linear DML, we need to pass `'backdoor.econml.dml.LinearDML'` to the `method_name` parameter. Notice that we also introduce an additional argument, `'discrete_treatment'`, and set it to `True`. This argument is passed to the model as a part of the `'init_params'` dictionary. We need to specify it because EconML DML estimators default to continuous treatment, while our treatment is binary:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dml.LinearDML',
    target_units='ate',
    method_params={
        'init_params': {
            # Define outcome and treatment models
            'model_y': LGBMRegressor(
                n_estimators=500, max_depth=10),
            'model_t': LogisticRegression(),
            # Specify that treatment is discrete
            'discrete_treatment': True
        },
        'fit_params': {}
    })
```

Let's predict on test data:

```
effect_pred = model.causal_estimator.effect(
    earnings_interaction_test.drop(['true_effect',
    'took_a_course'], axis=1))
effect_true = earnings_interaction_test['true_effect']
].values
```

Let's compute the error:

```
mean_absolute_percentage_error(effect_true, effect_pred)
```

This gives us the following:

```
0.0125345885989969
```

This error is roughly twice higher than the error for the linear DR-Learner.

Let's plot the results (*Figure 10.5*).

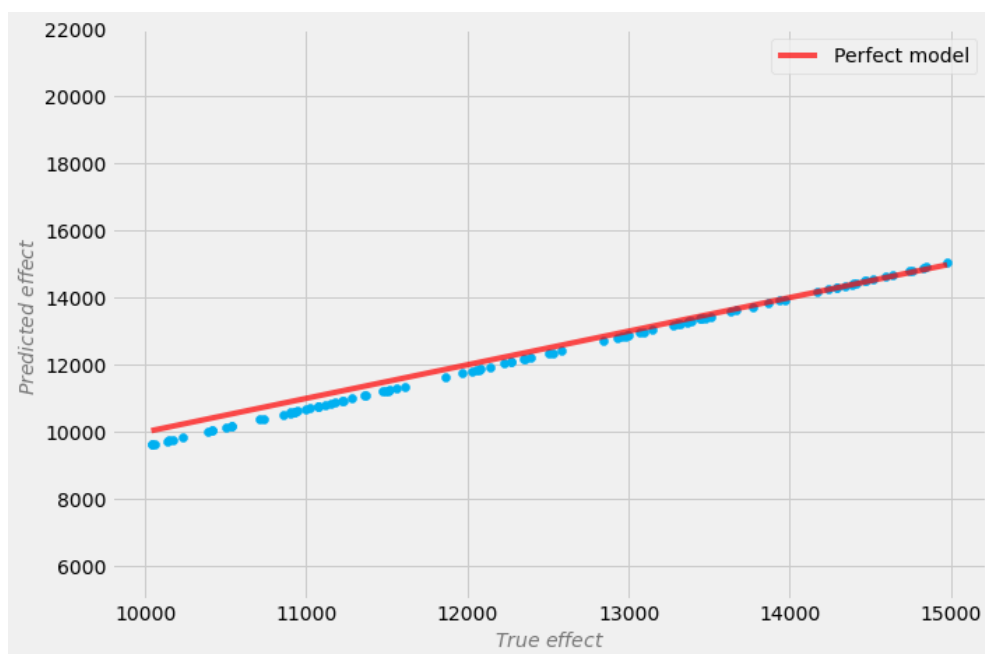


Figure 10.5 – The true versus predicted effect for linear DML

It seems that DML comes with a slightly biased estimate. In particular, the model underestimates the lower effect values (see the blue dots below the red line for lower values of the true effect in *Figure 10.5*).

Let's try to reduce the complexity of the outcome model and increase the number of cross-fitting folds:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dml.LinearDML',
    target_units='ate',
    method_params={
        'init_params': {
            # Define outcome and treatment models
            'model_y': LGBMRegressor(n_estimators=50,
                                     max_depth=10),
            'model_t': LogisticRegression(),
            # Specify that treatment is discrete
            'discrete_treatment': True,
            # Define the number of cross-fitting folds
            'cv': 4
        },
        'fit_params': {
        }
    })
```

Let's get predictions and calculate the error:

```
effect_pred = model.causal_estimator.effect(
    earnings_interaction_test.drop(['true_effect',
                                   'took_a_course'], axis=1))
effect_true = earnings_interaction_test['true_effect']
               .values
mean_absolute_percentage_error(effect_true, effect_pred)
```

This results in the following:

```
0.00810075098627887
```

That's better, but the results are still slightly worse than for the best DR-Learner.

Let's plot the results (*Figure 10.6*).

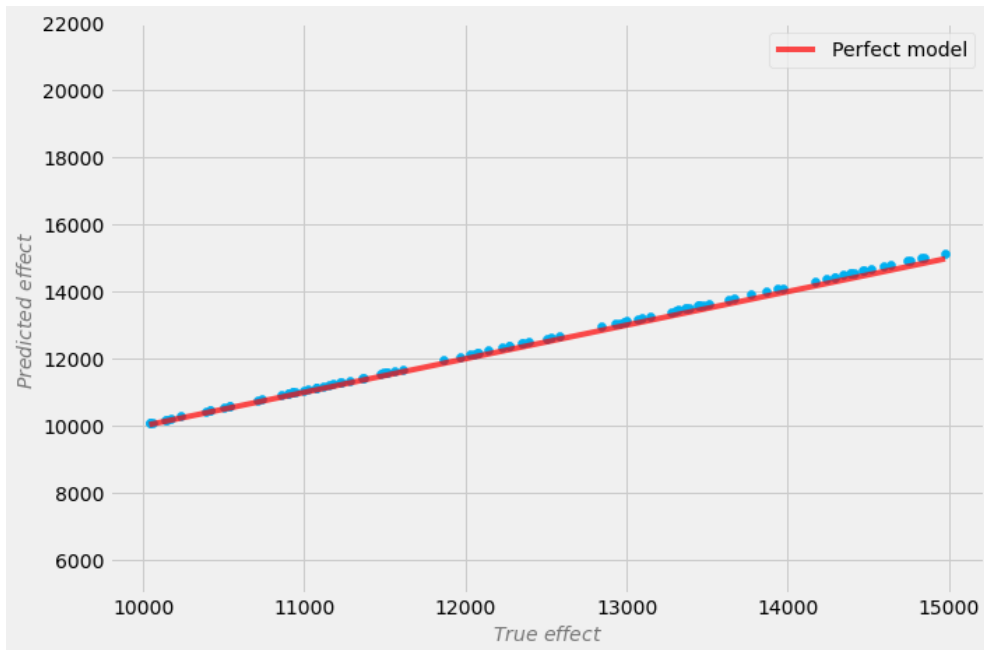


Figure 10.6 – The true versus predicted effect for linear DML with less complex models and four cross-fitting folds

These results look much better, but the model seems to systematically overestimate the effect.

Now, you might be puzzled about this whole process. Isn't there a fundamental challenge to all we're doing?

We changed some of the parameters of the base models and observed improved results – great – but this is only possible when we have simulated data, isn't it?

Is there anything we can do when simulated data is unavailable?

Hyperparameter tuning with DoWhy and EconML

Yes. One path we can take in order to optimize our models is to tune the hyperparameters of the outcome and treatment models.

Hyperparameter tuning can improve our estimators' fit to the data and their generalization to new instances (note that it cannot help with the estimand).

There are two main ways to run hyperparameter tuning with DoWhy and EconML.

The first is to wrap the models in one of the sklearn cross-validation classes, `GridSearchCV`, `HalvingGridSearchCV`, or `RandomizedSearchCV`, and pass the wrapped models into the constructor.

Let's try it!

This time, we'll use the LGBM classifier instead of logistic regression to predict the treatment. We hope that with some hyperparameter tuning, we can outperform the model with logistic regression.

Now's also a good moment for a reminder. In DML, the treatment model does not estimate propensity scores but rather predicts treatment values directly. In this setting, probability calibration – important for propensity scores – is not that critical and so using a more complex model might be beneficial, despite the fact that its probabilities might be less well calibrated than in the case of logistic regression.

Ready to code?

We'll start by defining our outcome (`model_y`) and treatment (`model_t`) models and wrapping them in a grid search wrapper:

```
model_y = GridSearchCV(
    estimator=LGBMRegressor(),
    # Define the model's parameter search space
    param_grid={
        'max_depth': [3, 10, 20, 100],
        'n_estimators': [10, 50, 100]
    },
    # Define GridSearch params
    cv=10, n_jobs=-1, scoring='neg_mean_squared_error'
)

model_t = GridSearchCV(
    estimator=LGBMClassifier(),
    # Define the model's parameter search space
    param_grid={
        'max_depth': [3, 10, 20, 100],
        'n_estimators': [10, 50, 100]
    },
    # Define GridSearch params
    cv=10, n_jobs=-1, scoring='accuracy'
)
```

We set `GridSearchCV`'s `cv` parameter to 10. This parameter determines how many folds will be used to tune the hyperparameters of our treatment and outcome models. Note that this is different from the DML estimator's `cv` parameter, which controls the number of folds for cross-fitting. It's good to remember this distinction as both parameters control different functionalities.

Now, let's estimate the effects. Doing so is straightforward. We simply pass wrapped `model_y` and `model_t` objects to the `init_params` dictionary:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dml.LinearDML',
    target_units='ate',
    method_params={
        'init_params': {
            # Pass models wrapped in GridSearchCV objects
            'model_y': model_y,
            'model_t': model_t,
            # Set discrete treatment to `True`
            'discrete_treatment': True,
            # Define the number of cross-fitting folds
            'cv': 4
        },
        'fit_params': {
        }
    })
```

After computing the results, we obtain the following:

```
0.00179346825212699
```

The MAPE value is only 0.17%. This is much better than anything we've seen so far!

In particular, this result is more than 3.5 times better than the result of our best DR-Learner.

Let's examine the results visually (*Figure 10.7*).

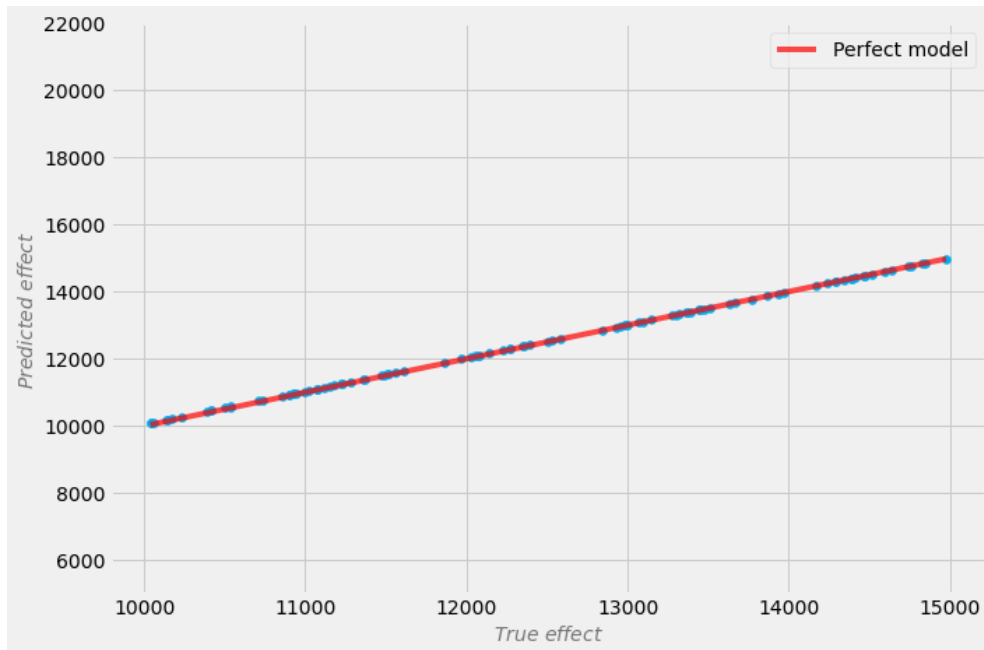


Figure 10.7 – DML results after hyperparameter tuning

These results look very good!

The tuned model did a really good job! To the extent that such a small error might seem unrealistic.

When looking at *Figure 10.7*, we should remember two things. First, we’re working with a relatively simple synthetic dataset, which should be easily solvable by a powerful enough model. Second, for comparability, we keep the same value ranges for both axes in the plot that we used for the previous models. This (to an extent) hides the fact that the model has *some error*.

If you see results like this in real life, don’t forget about **Twyman’s law**. The law has multiple variants (Kohavi et al., 2020) and states that any figure or statistic that looks interesting is most likely wrong. Twyman’s law can be a valuable reminder to retain healthy skepticism in our work.

As we’ve just seen, tuning hyperparameters for CATE models can bring significant improvements. At the beginning of this section, we said that there are two ways to tune hyperparameters with DoWhy and EconML.

Let’s discuss the second way now.

The second way to tune hyperparameters is to do it in advance using your favorite hyperparameter tuning framework, such as Hyperopt or Optuna, and then simply use the best models found.

There are two main advantages to this approach. The first is that you're *not limited* to the grid search or random search options available in sklearn. With Hyperopt, Optuna, and some other frameworks, you can leverage the power of more efficient Bayesian optimization.

Note that the cross-validation procedure or a separate validation set should be used when tuning the hyperparameters in order to minimize the risk of leakage and overfitting. The second reason to tune your parameters independently – perhaps even more important – is that while passing sklearn wrappers to EconML, only a subset of the data will be used for tuning. This might be statistically less stable.

As an additional remark, I want to comment once again on *cross-validation*. In *Chapter 7*, we said that *cross-validation* is not a suitable technique to validate structural aspects of causal models, yet it can be useful for validating the estimation part.

This also includes hyperparameter tuning for base estimators that we carried out in this section.

The idea of hyperparameter tuning brings more interesting questions. For instance, if the outcome model is tuned and trained on a particular dataset, how will it perform on the data that is out of distribution? Wouldn't it be reasonable to expect that a causal model would handle this task smoothly? At the end of the day, we've learned about a causal structure that goes beyond associations in the dataset, right?

The answer depends on a number of factors. Let's discuss two important ones.

First, the answer depends on which models we use. For instance, tree-based models *do not extrapolate* beyond the maximum value in the training set. Let's imagine that you only have one predictor in your problem, X , and all values of X in your training dataset are bounded between 0 and 1,000. Additionally, let's assume that the outcome, Y , is always two times X .

This is a very simple problem, but if you test your tree-based model on data where X ranges from 1,000 to 2,000 (that is, all values are outside the training range), the prediction for all observations will be a constant value that is no greater than the maximum outcome seen in the training dataset (which would be $1000 \times 2 = 2000$).

This behavior comes from the fact that trees learn a set of rules from the data (e.g., if $4.5 < x < 5.5$, then $y = 10$) and they would need to essentially *make up* the rules for data that is outside the training range. The fact that we're using a causal DML model will not surpass this limitation.

If we use a linear regression model or a neural network, on the other hand, these models will extrapolate. This brings us to the second factor.

The extrapolation can only be correct under certain limited circumstances – when the relationship between X and Y is linear or, in the case of neural networks, when the activation function is *similar* to the target function (Xu et al., 2021).

In this scenario, using DML will also not surpass the limitations of the outcome and treatment models.

These limitations are extremely important in any production system and should be taken into account at the early stages of the system design.

Is DML a golden bullet?

We just discussed some of the general limitations of machine learning models. Accepting them as they are, let's take a closer look at the DML framework itself.

DML is a great and flexible method and tends to work very well in a broad set of circumstances; yet – like any other method – it has its own set of constraints. We will discuss the main ones here.

We already know that one of the assumptions behind DML is a lack of hidden confounding. In their 2022 paper, Paul Hünermund and colleagues showed that not meeting this assumption leads to significant biases that are similar in magnitude to biases that we get from other, much simpler methods, such as LASSO (Hünermund et al., 2022). These findings are consistent with the results obtained by Gordon et al. (2022) in a large-scale study that benchmarked DML using high-dimensional non-experimental data from the Facebook Ads platform. Note that DML is sensitive to not only unobserved common causes of treatment and outcomes but also other bad control schemes that allow for non-causal information flow in the graph. This is also true for other causal models (for a comprehensive overview of bad control schemes, check out Cinelli et al., 2022).

While it might not be particularly surprising to some of you that DML can lead to biased results when hidden confounding cannot be excluded, there are two ideas I'd like to highlight here.

It can be tempting for some researchers and practitioners to transfer their insights from the fields of deep learning and big data to the domain of causal inference. Let's discuss why such an approach might backfire.

First, those familiar with deep learning will recognize that in certain cases, increasing the training sample size can lead to significant model performance improvements. In causal machine learning, this approach can also be helpful when the base models do not have enough data to learn a useful mapping or when the original data does not provide enough diversity to satisfy the positivity assumption. That said, when these conditions are met, adding more observations *cannot help* in reducing *causal bias* that arises from *unmet causal assumptions*.

The reason for this is that causal bias comes from ill-defined structural relationships between variables, not from an insufficient amount of information within a dataset. In other words, causal bias is related to the *estimand* misspecification and not to the statistical *estimate*.

Secondly, sometimes, in particular in big data settings, practitioners of traditional machine learning might benefit from adding more features to the model. For many tech companies, it's not unusual to use hundreds or even thousands of variables for predictive tasks. The logic behind adding more features to a statistical model relies on the assumption that having more features translates to more predictive power, which in turn translates to better predictions. This reasoning might lead to beneficial outcomes when using modern machine learning techniques.

The causal setting is different. As we know, adding arbitrary features to the model might result in opening non-causal paths in the DAG. This can introduce confounding and lead to biased estimates of causal parameters.

In practice, it might be better to include a smaller set of variables in your model and make sure that they do not introduce unwanted confounding. This might be easier said than done, yet in cases where it's unclear which variables to include, this approach can be treated as a hypothesis-generating process that can be iteratively improved over time.

If you cannot exclude the possibility of unobserved confounding in your setting and you cannot afford to perform an experiment (for ethical, financial, or other reasons), DML can still be useful for you. Chernozhukov et al. (2022) proposed a general framework for finding confidence bounds under unobserved confounding. The method, belonging to a broader family of sensitivity analysis methods, allows us to assess how stable the effect would be under hidden confounding of different strengths.

This solution can bring a lot of value to business decision-making, providing us with potentially actionable insights in light of difficult or impossible-to-meet assumptions.

Doubly robust versus DML

Before we conclude this section, let's make a quick comparison between DR-Learner and DML. Both methods can lead to good results. Both offer smart solutions for reducing the bias and variance of causal estimates. After reading this chapter, you might wonder: which one should I choose in practice?

I cannot give you a definitive answer that will guarantee that one model always outperforms the other, but we can look into some recommendations together. Hopefully, these recommendations will help you make decisions in your own use cases.

The first key difference between the models is that DML works for categorical and continuous treatments, while DR-Learner (similarly to TMLE) is – by design – limited to categorical treatments. This might sort some stuff out for you right away.

Now, let's focus on the particular implementations available in EconML. As we said before, `DRLearner` is more than just a simple DR estimator as it uses orthogonalization and cross-fitting under the hood. In a sense, you can think of `DRLearner` as a DR estimator on steroids as it incorporates various modern techniques in order to minimize error. One thing that remains unchanged is that DR correction is at the heart of the method. That's one of the reasons why it behaves differently compared to EconML's DML methods.

The main advantage of the DR method is that the error of the final estimate is only affected by the product of errors of the outcome model and the treatment model (Hernan & Robins, 2020). DR methods might perform better whenever the outcome model is misspecified.

On the other hand, DR-Learner will usually have higher variance than DML. Jane Huang from Microsoft suggests that this might become particularly noticeable when “*there are regions of the control space (...) in which some treatment has a small probability of being assigned*” (Huang et al., 2020). In this scenario, “*DML method could potentially extrapolate better, as it only requires good overlap on average to achieve good mean squared error*” (Huang et al., 2020). Note that TMLE could also perform well in the latter case. Another setting where DML might outperform DR-Learner is under sparsity in a high-dimensional setting, as demonstrated by Zachary Clement (Clement, 2023; note that in this case, DR-Learner’s results could be potentially improved by adding lasso regularization to the final model).

In addition, a comparison between DML and TMLE might interest you. As we mentioned earlier, a detailed discussion on this topic is beyond the scope of this book, yet great resources are available. To start, check out van der Laan & Hejazi (2019) and Díaz (2020).

What’s in it for me?

It’s difficult to point out just one algorithm that would be guaranteed to unconditionally work better than other solutions.

That said, given the results of our experiments in this and the previous chapter, we might get the impression that DML outperforms other methods. This impression is somewhat biased as we did not tune hyperparameters for the remaining methods. On the other hand, DML is often recommended as the go-to method for continuous treatments. DR methods cannot compete in this category. S-Learner can be adapted to work with continuous treatment, but the current version of EconML does not support this feature.

If you cannot benchmark a wide array of methods and you’re convinced that your data contains heterogeneous treatment effects, my recommendation would be to start with S-Learner, in particular if computational resources are an issue. T-Learner and X-Learner might be good to add to the mix if your treatment is discrete.

If computational resources are not an issue, DML (continuous or discrete treatment) and DR-Learner or TMLE (discrete treatment) will usually be good shots. That said, it’s always good to have a simpler model as a benchmark. It’s very easy to plug in complex base estimators to DML or DR/TMLE and get carried away, not knowing that the complexity pulls us far away from the best results we could achieve.

In the next section, we’ll discuss one more family of methods that might be worth considering.

Before this, let’s summarize the current section.

In this section, we introduced DML. We discussed the key concepts that constitute the uniqueness of the method: orthogonalization and cross-fitting. We demonstrated how to perform hyperparameter tuning for submodels using DoWhy and EconML. Then, we discussed the weaknesses of DML, finishing with a comparison between DR-Learner and DML.

Causal Forests and more

In this short section, we'll provide a brief overview of the idea behind **Causal Forests**. We'll introduce one of the EconML classes implementing the method. An in-depth discussion on Causal Forests and their extensions is beyond the scope of this book, but we'll point to resources where you can learn more about forest-based causal estimators.

Causal Forest is a tree-based model that stems from the works of Susan Athey, Julie Tibshirani, and Stefan Wager (Wager & Athey, 2018; Athey et al., 2019). The core difference between regular random forest and Causal Forest is that Causal Forest uses so-called **causal trees**. Otherwise, the methods are similar and both use resampling, predictor subsetting, and averaging over a number of trees.

Causal trees

What makes causal trees different from regular trees is the **split criterion**. Causal trees use a criterion based on the estimated treatment effects, using so-called **honest splitting**, where the splits are generated on training data, while leaf values are estimated using a hold-out set (this logic is very similar to cross-fitting in DML). For a more detailed overview, check out Daniel Jacob's article (Jacob, 2021), Mark White's blog post (<https://bit.ly/CausalForestIntro>), or the chapter on Causal Forests in White & Green (2023). For a high-level introduction, check out Haaya Naushan's blog post (<https://bit.ly/CausalForestHighLvl>). For a deep dive, refer to Wagner & Athey (2018) or Athey et al. (2019).

Forests overflow

EconML offers a wide variety of estimators that build on top of the idea of Causal Forests. The methods share many similarities but may differ in significant details (for instance, how first-stage models are estimated). This might translate to significant differences in computational costs. To understand the differences between the different estimator classes, check out the EconML documentation page here: <https://bit.ly/EconMLCausalForestDocs>.

Advantages of Causal Forests

Causal Forest models are a good choice when dealing with high-dimensional data. They provide valid confidence intervals while being non-parametric and offering high flexibility.

To start with Causal Forests, the `CausalForestDML` class from the `dml` module will likely be the best starting point in most cases. EconML also offers a *raw* version of Causal Forest – `CausalForest` – which can be found in the `grf` module. The latter class does not estimate the *nuisance parameter*. You might want to use this basic implementation in certain cases, but keep in mind that this might lead to suboptimal results compared to `CausalForestDML`.

Causal Forest with DoWhy and EconML

To use Causal Forest with DML orthogonalization, we simply pass the method's name to the `method_name` parameter. You might also want to make your own choice of the outcome and treatment models. This is done exactly in the same way as we did for the DML estimator. If your treatment is discrete, don't forget to include `discrete_treatment': True` in the `init_params` dictionary. Here's a code example for reference:

```
estimate = model.estimate_effect(
    identified_estimand=estimand,
    method_name='backdoor.econml.dml.CausalForestDML',
    target_units='ate',
    method_params={
        'init_params': {
            'model_y': LGBMRegressor(n_estimators=50,
                                     max_depth=10),
            'model_t': LGBMClassifier(n_estimators=50,
                                     max_depth=10),
            'discrete_treatment': True,
            # Define the num. of cross-fitting folds
            'cv': 4
        },
        'fit_params': {
        }
    }
)
```

This model's error is around 4.6% – a result comparable to S- and X-Learners. Note that we did not tune hyperparameters for this model.

Figure 10.8 presents the results of the model:

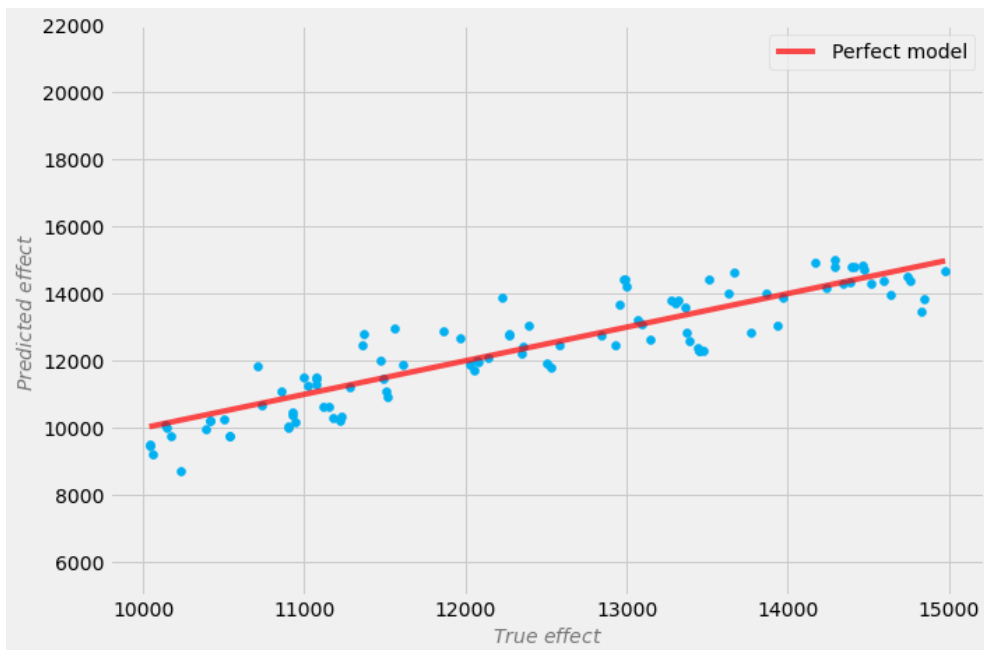


Figure 10.8 – Results for Causal Forest with DML (untuned)

Before we conclude this section, let's summarize the results of all CATE models on the machine learning earnings interaction dataset from *Chapter 9* and *Chapter 10* in a single table. *Table 10.1* presents this summary:

Estimator	MAPE
S-Learner	5.02%
T-Learner	8.13%
X-Learner	3.63%
Linear DR-Learner	0.62%
Linear DML	1.25%
Linear DML (tuning)	0.17%
Causal Forest	4.60%

Table 10.1 – A summary of the results for all models from Chapter 9 and Chapter 10 on the machine learning earnings interaction dataset

In this section, we discussed the core differences between classic random forest and Causal Forests. We said that the main difference between the two algorithms lies in how the trees are constructed. We pointed to resources where you can learn more about methods based on Causal Forests. Finally, we implemented `CausalForestDML` – a method that extends the basic Causal Forest with nuisance parameter estimation – and summarized the results for all CATE models on the machine learning earnings interaction dataset.

In the next section, we'll discuss estimating heterogeneous treatment effects from experimental data. Ready?

Heterogeneous treatment effects with experimental data – the uplift odyssey

Modeling treatment effects with experimental data is usually slightly different in spirit from working with observational data. This stems from the fact that experimental data is assumed to be unconfounded by design (assuming our experimental design and implementation were not flawed).

In this section, we'll walk through a workflow of working with experimental data using EconML. We'll learn how to use EconML's basic API and see how to work with discrete treatments that have more than two levels. Finally, we'll use some causal model evaluation metrics in order to compare the models.

The title of this section talks about heterogeneous treatment effects – we already know what they are, but there's also a new term: *uplift*. **Uplift modeling** and **heterogeneous (aka conditional) treatment effect modeling** are closely related terms. In marketing and medicine, *uplift* simply means the quantity of change in some outcome in a treatment group (or subject) as compared to a control group – in other words, the treatment effect. Although uplift can be calculated for an entire treatment group, it can also be estimated for individual subjects or subgroups. In the latter case, it's synonymous with conditional (heterogeneous) treatment effects.

The data

In this section, we'll use the data from Kevin Hillstrom's *MineThatData* challenge (<https://bit.ly/KevinsDataChallenge>).

Before we start, I want to take a moment to express my gratitude for Kevin's generosity. Kevin agreed that we could use his dataset in the book. I appreciate this and believe that Kevin's decision will help us all become better causal data scientists, one step at a time. Thank you, Kevin!

Now, let's understand how the dataset is structured. The data comes from a randomized email experiment on 64,000 customers.

The treatment has three levels and was administered randomly:

- 1/3 of the customers received an email campaign featuring men's merchandise (treatment 1)
- 1/3 received an email campaign featuring women's merchandise (treatment 2)
- 1/3 received no campaign (control)

The features include the time since the last purchase (*recency*), the amount of dollars spent in the past year (*history*), indicators of the type of merchandise bought in the past year (*mens* or *womens*), an indicator of whether the customer was new in the past 12 months (*newbie*), what channel they purchased from previously (*channel*), and what type of area they live in (*rural*, *suburban*, *urban*; *zip_code*). Treatment is a three-level discrete variable describing which email campaign the customer received (*men's*, *women's*, or *control*). Finally, we have three outcome variables: *visit*, *conversion*, and *spending*.

We'll use *spending* as our target. It records a customer's spending within the two weeks following the delivery of the email campaign.

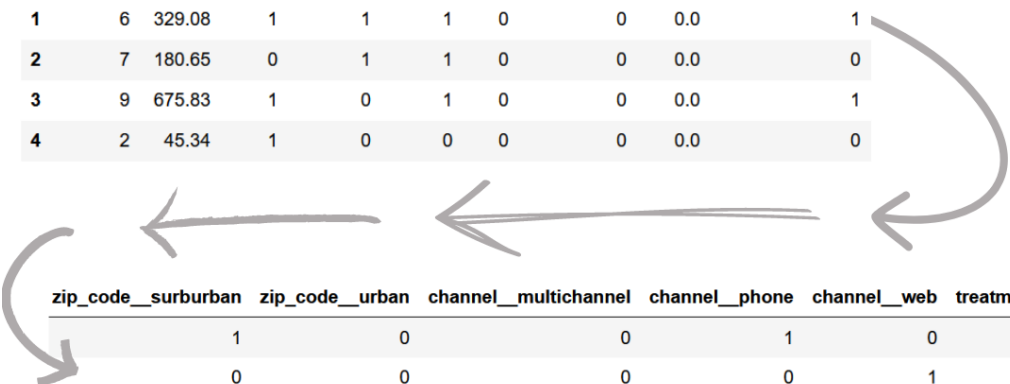
Let's read in the data.

We'll read the main dataset from a CSV file and the treatment mapping from a JSON file:

```
hillstrom_clean =  
    pd.read_csv(r'./data/hillstrom_clean.csv')  
  
with open(r'./data/hillstrom_clean_label_mapping.json',  
        'r') as f:  
    hillstrom_labels_mapping = json.load(f)
```

Our treatment has three levels. I stored the mapping in a dictionary serialized as a JSON object.

Let's see a couple of rows from our dataset (*Figure 10.9*):



	recency	history	mens	womens	newbie	visit	conversion	spend	zip_code__rural
0	10	142.44	1	0	0	0	0	0.0	0
1	6	329.08	1	1	1	0	0	0.0	1
2	7	180.65	0	1	1	0	0	0.0	0
3	9	675.83	1	0	1	0	0	0.0	1
4	2	45.34	1	0	0	0	0	0.0	0

	zip_code__surburban	zip_code__urban	channel__multichannel	channel__phone	channel__web	treatment
	1	0	0	1	0	1
	0	0	0	0	1	0
	1	0	0	0	1	1
	0	0	0	0	1	2
	0	1	0	0	1	1

Figure 10.9 – The first five rows of the Hillstrom dataset

We cut the display in *Figure 10.9* into two parts for readability. As you can see, there are many sparse binary features in the data. Roughly half of them are one-hot-encoded multi-level categorical features: zip code area and channel.

The *zip code* and *channel* variables are represented as fully one-hot-encoded sets of variables. This means that for each row, the set of variables representing a channel (or zip code area) will have exactly one column with the value 1.

This makes one of the columns redundant as its value can be unambiguously inferred from the values of the remaining columns in the set (if `channel__web` and `channel__phone` are both 0, `channel__multichannel` has to be 1 – by definition of one-hot-encoding).

Note that this setting with redundant information will introduce multicollinearity into the data. Although this might not be a problem for the tree-based methods, it will impact linear models' convergence and performance.

To address this in advance, let's drop the redundant columns:

```
hillstrom_clean = hillstrom_clean.drop(['zip_code__urban',
    'channel__web'], axis=1)
```

The data is technically ready.

Let's examine its properties.

Testing the waters – how unconfounded are you, my dear?

We expect that experimental data is unconfounded due to randomization, yet sometimes experimental data might be biased. This can be the case for a number of reasons: technical glitches, invalid procedures, treatment leakage, and more.

We cannot fully test whether randomization was valid (there is no easy way to spot arbitrary unobserved confounding), but we can test whether observed variables can predict treatment. If this were the case, we'd have an indicator that the randomization or data collection process was flawed.

One practical way to perform this check is to split your data in half: train a model that predicts the treatment from covariates on one half of the data and predict treatment values on the other half. Our model's performance should essentially be random.

Let's implement this process.

First, let's split the data into treatment, outcome, and feature vectors:

```
hillstrom_X = hillstrom_clean.drop(['visit', 'conversion',
    'spend', 'treatment'], axis=1)
hillstrom_Y = hillstrom_clean['spend']
hillstrom_T = hillstrom_clean['treatment']
```

Second, let's check whether the treatments are distributed uniformly in our data as expected:

```
sample_size = hillstrom_clean.shape[0]
hillstrom_T.value_counts() / sample_size
```

We get the following output:

```
1    0.334172
2    0.332922
0    0.332906
Name: treatment, dtype: float64
```

All values are close to 33.3%, which indicates that the treatments were distributed uniformly.

Third, let's perform a train-test split for the randomness checks:

```
X_train_eda, X_test_eda, T_train_eda, T_test_eda =
    Train_test_split(hillstrom_X, hillstrom_T_eda,
        test_size=.5)
```

Let's verify the split's quality. We expect that roughly 33% of observations should be assigned to each treatment group:

```
T_test_eda.value_counts() / T_test_eda.shape[0]
```

We get the following:

```
0    0.335156
2    0.333250
1    0.331594
Name: treatment, dtype: float64
```

This looks good!

Fourth, let's fit the classifier that aims to predict treatment T from features X :

```
lgbm_eda = LGBMClassifier()
lgbm_eda.fit(X_train_eda, T_train_eda)
```

One side remark that I want to share with you here is that, in general, for the LGBM classifier, it might be better for us not to one-hot encode our categorical features (such as zip area or channel). The model handles categorical data natively in a way that can improve the results over one-hot-encoded features in certain cases.

That said, we'll continue with one-hot-encoded features as they give us more flexibility at later stages (other models might not support categorical features natively) and we don't expect significant improvements in our case anyway.

Fifth, let's predict on the test data and calculate the accuracy score:

```
T_pred_eda = lgbm_eda.predict(X_test_eda)
accuracy_score(T_test_eda, T_pred_eda)
```

If there's no confounding in the data, we expect to get an accuracy of around 33%. The result is as follows:

```
0.33384375
```

This looks good!

Finally, we can generate empirical confidence intervals to see whether this score falls within them:

```
random_scores = []

test_eda_sample_size = T_test_eda.shape[0]

for i in range(10000):
    random_scores.append(
```

```
(np.random.choice(
    [0, 1, 2],
    test_eda_sample_size) == np.random.choice(
    [0, 1, 2],
    test_eda_sample_size)).mean())

np.quantile(random_scores, .025), np.quantile(random_scores, .975)
```

We compare how often two random vectors of three mutually exclusive values are the same. For each draw, we compare as many vectors as we have observed in our test set. We repeat this procedure 10,000 times and calculate 0.025 and 0.975 quantiles. This gives us 95% confidence intervals:

```
(0.32815625, 0.33850078125)
```

The accuracy score that we obtained for our model lies within the boundaries of these intervals. This gives us more confidence that the data is not observably confounded.

See *Figure 10.10* for reference.

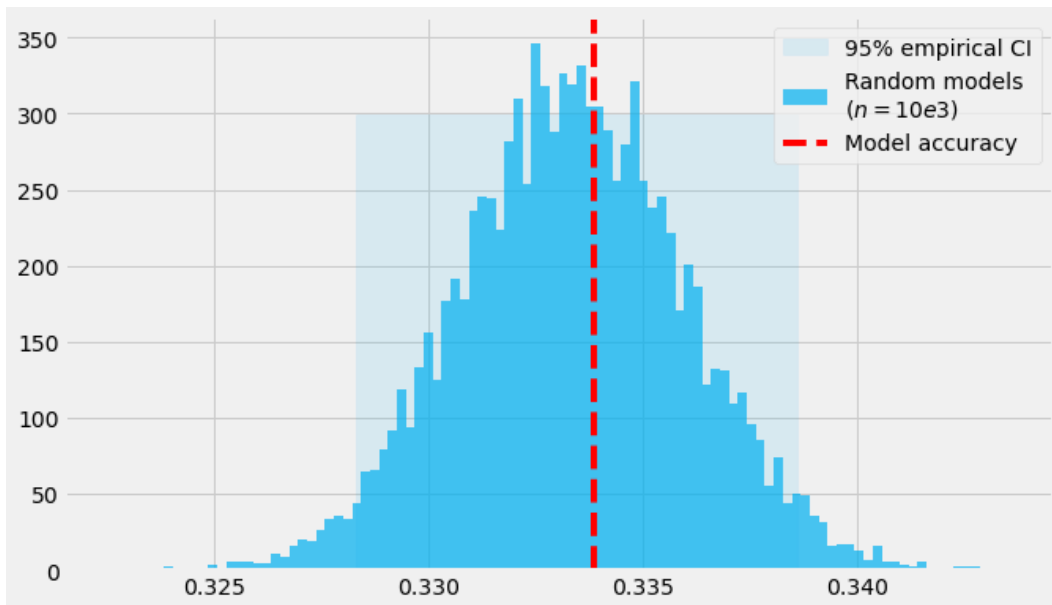


Figure 10.10 – Empirical distribution of random models versus model accuracy

Another similar way to check randomization under observed variables is to use a **classifier two-sample test (C2ST)**; Diemert et al., 2018; Lopez-Paz & Oquab, 2016). The procedure is very similar, with the difference that the C2ST compares the *loss* of a classifier trained on the actual data with the distribution of losses of a classifier predicting random noise, instead of comparing accuracies.

C2STs based on different types of classifiers might perform better in certain cases or for certain data types (e.g., images). Check out Lopez-Paz & Oquab (2016) for details.

Choosing the framework

We started this section with the statement that modeling treatment effects in experimental data differs from working with observational data.

We already got a taste of some of the differences in the workflow. Instead of starting the process by defining a graph, we performed randomization checks. The fact that we do not explicitly define a graph means that we won't explicitly look for the estimand and that there's no clear basis for performing refutation tests.

These differences make it a natural choice to work with EconML estimators directly (rather than using DoWhy's four-step process) when we use experimental data.

We're almost ready to start, but before we open the toolbox, let me share something with you.

We don't know half of the story

I was recently skimming through a paper discussing *selection models*. These models are used by statisticians who perform meta-analyses to assess how much the effect of interest might be skewed due to research that was *not* published.

As researchers and practitioners, we are used to working with materials demonstrating successful outcomes. Research papers and blog posts rarely focus on challenges and failures and not many journals (though there are exceptions) are willing to publish negative results. Though understandable from a psychological standpoint, this bias toward discussing successful results has its price.

Those of us who've worked with meta-analyses know about the tools we use in order to assess **publication bias**. Those of us working in the industry will have perhaps seen junior data scientists surprised by the messiness of real-world data, something they never found in their handbooks and courses. Moreover, most of us have likely been in a similar position at least once – surprised at how far away real-world data can be from our expectations.

This surprise comes partially from the fact that the world is complex, yet the way we structure incentives in the education system and the publish-or-perish culture also play a role here.

In this section, we'll see how things should look when everything works smoothly, but we'll also see what happens when they don't.

I hope this approach will be beneficial to you.

Now, let's get some more context and begin our journey into the wild!

Kevin's challenge

As we said in the beginning, the data that we're about to use comes from an online experiment, but this is only half of the story. The data was released as part of a challenge organized by Kevin back in 2008. The challenge also included a series of questions, among others – *if you could eliminate 10,000 customers from the campaign, which 10,000 should that be?*

This and the other questions posed by Kevin are very interesting and I am sure they stir up a sense of excitement in anyone interested in marketing.

Check out the full list of Kevin's questions here: <https://bit.ly/KevinsDataChallenge>.

Although most, if not all, of these questions can be answered with the tools we'll use in this section (plus some clever analytics), we'll ask a simpler question instead: which of the models would help us make decisions that would translate to the best financial results?

The fact that this question might seem simpler does not make our endeavor easier.

There are a number of challenges that we'll need to overcome:

- First, we have more than one treatment. This makes model evaluation more challenging than in a binary case.
- Second, although the dataset is relatively large (64,000 observations), the conversion rate is low.

Let's take a look at the label mapping to find out how to work with the treatment:

```
hillstrom_labels_mapping
```

This gives us the following:

```
{'control': 0, 'womans_email': 1, 'mens_email': 2}
```

0 represents the control group assignment, 1 represents women's emails, and 2 represents men's emails.

Let's check how many people bought something under both treatments:

```
(hillstrom_Y[hillstrom_T > 0] > 0).sum()
```

We get the following:

```
456
```

This is roughly 0.7% of the entire dataset.

Let's open the toolbox!

Opening the toolbox

In this section, we'll use six algorithms: S-Learner, T-Learner, X-Learner, DR-Learner, linear DML, and Causal Forest with DML. We'll compare their performance from three different angles: computational cost, ability to sort true uplift, and average expected outcome. If some of these sound cryptic to you, don't worry; we'll explain them on the way.

Let's start by instantiating the models. First, we'll create a function that returns an instance of the LGBM model to make the code a bit more readable:

```
def create_model(model_type, n_estimators=100, max_depth=10, learning_rate=.01):
    if model_type == 'regressor':
        return LGBMRegressor(
            n_estimators=n_estimators,
            max_depth=max_depth,
            learning_rate=learning_rate)
    elif model_type == 'classifier':
        return LGBMClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            learning_rate=learning_rate)
    else:
        raise NotImplementedError(
            f'Model type `{model_type}` not implemented.')
```

The function returns an instance of an LGBM regressor or LGBM classifier, depending on the argument we pass to the `model_type` parameter.

Let's instantiate our causal models:

```
s_learner = SLearner(
    overall_model=create_model('regressor')
)

x_learner = XLearner(
    models=[
        create_model('regressor'),
        create_model('regressor'),
        create_model('regressor'),
    ],
    cate_models=[
        create_model('regressor'),
        create_model('regressor'),
        create_model('regressor'),
    ],
```

```

    ]
)

t_learner = TLearner(
    models=[
        create_model('regressor'),
        create_model('regressor'),
        create_model('regressor'),
    ]
)

dml = LinearDML(
    model_y=create_model('regressor'),
    model_t=create_model('classifier'),
    discrete_treatment=True,
    cv=5
)

dr = DRLearner(
    model_propensity=LogisticRegression(),
    model_regression=create_model('regressor'),
    model_final=create_model('regressor'),
    cv=5,
)

cf = CausalForestDML(
    model_y=create_model('regressor'),
    model_t=create_model('classifier'),
    discrete_treatment=True,
    cv=5
)

```

This is very similar to what we were doing before when using DoWhy wrappers. The main difference is that we now pass model parameters directly to model constructors rather than encoding them in an intermediary dictionary.

Note that for linear DML and Causal Forest, we set `discrete_treatment` to `True`. We don't do so for meta-learners and DR-Learner because these models only allow discrete treatments (S-Learner can be generalized to continuous treatments, but the current version of EconML does not support that). Also note that our `create_model()` function returns estimators with the same set of pre-defined parameters for each base learner.

First things first

We want to assess the performance of our models and so we'll divide our data into training and test sets.

As a reminder – we won't be able to compute an error metric like we'd do in the case of traditional supervised learning. We'll need to use different approaches, but having a test set will still be beneficial for us.

We'll use scikit-learn's `train_test_split` function to perform the split:

```
X_train, X_test, y_train, y_test, T_train, T_test =  
    train_test_split(  
        hillstrom_X,  
        hillstrom_Y,  
        hillstrom_T,  
        test_size=.5  
    )
```

We set the test size to 0.5. The reason I chose this value is that although our dataset has 64,000 observations, only a tiny fraction of subjects actually converted (made a purchase). Let's see how many conversion instances we have in each of the splits:

```
(y_train[T_train > 0] > 0).sum(),  
(y_test[T_test > 0] > 0).sum()
```

This gives us the following:

```
(227, 229)
```

That's only 227 converted observations in training and 229 in the test set. This is not a very large number for machine learning methods. The trade-off here is between having enough observations to effectively train the models and having enough observations to effectively evaluate the models. Our case is pretty challenging in this regard.

Nonetheless, let's fit the models and see how much we can get out of them.

We'll fit the models in a loop to save ourselves some space and time. Speaking of time, we'll measure the fitting time for each of the algorithms to get a sense of the differences between them in terms of computational cost.

Let's start by creating a dictionary that aggregates all the models:

```
models = {  
    'SLearner': s_learner,  
    'TLearner': t_learner,  
    'XLearner': x_learner,  
    'DRLearner': dr,
```

```

    'LinearDML': dml,
    'CausalForestDML': cf
}

```

Let's iterate over the dictionary and fit the models:

```

for model_name, model in models.items():
    start = time.time()
    model.fit(
        Y=y_train,
        T=T_train,
        X=X_train,
        inference='bootstrap'
    )
    stop = time.time()
    print(f'{model_name} fitted in {stop - start:0.4f}
          seconds.')

```

This gives us the following output:

```

Slearner fitted in 0.1730 seconds.
Xlearner fitted in 0.8588 seconds.
Tlearner fitted in 0.2894 seconds.
DRlearner fitted in 2.0550 seconds.
LinearDML fitted in 4.0384 seconds.
CausalForestDML fitted in 6.3661 seconds.

```

From this output, we can get a sense of the computational costs related to each model. It would be great to repeat this experiment a couple of times to get more reliable estimates of the differences between the models. I did this in advance for you. *Table 10.2* contains rough estimates of how much computational time each of the methods needs compared to the S-Learner baseline:

Model	Time (multiplier compared to S-Learner)
S-Learner	1x
T-Learner	2x
X-Learner	5x
DR-Learner	13x
LinearDML	27x
CausalForestDML	39x

Table 10.2 – Relative training times of six different EconML CATE estimators

In *Table 10.2*, the right column displays the time taken by each model to complete its training in comparison to the S-Learner baseline. For instance, *2x* means that a model needed twice as much time as S-Learner to finish the training.

All the models were trained on the three-level treatment scenario, using the same dataset, with identical base estimators and default hyperparameters.

T-Learner needs twice as much training time as S-Learner, and Causal Forest with DML needs a stunning 39 times more time! Just to give you an understanding of the magnitude of this difference: if S-Learner trained for 20 minutes, Causal Forest would need 13 hours!

Computational cost becomes an important factor whenever we design a production-grade system. Causal Forest with DML might easily become prohibitively expensive on very large datasets, especially if we add hyperparameter tuning to the mix.

Our models are trained. Let's think about how to evaluate them.

Uplift models and performance

There's a significant body of literature on evaluating uplift models. The most popular metrics include the **Qini coefficient** (Radcliffe, 2007) and the **Area Under the Uplift Curve (AUUC)** (Rzepakowski & Jaroszewicz, 2010).

The Qini coefficient is a single-scalar model quality summary that is based on a comparison between the Qini curve for an actual model and a random model. The Qini curve shows the cumulative number of positive outcomes scaled by the number of treated units.

The AUUC is based on a similar idea – it measures the area under the cumulative uplift curve. We plot uplift on the y axis against the percentage of observed units (the cumulative percentage of our sample size) *sorted by* model predictions on the x axis and calculate the area under the curve.

The AUUC and Qini are very popular and you can find many open source implementations for these metrics (for instance, in the `uplift-analysis` package: <https://bit.ly/UpliftAnalysis>). Both metrics were originally designed for scenarios with binary treatments and binary outcomes. Another popular choice for uplift model performance assessment is the **uplift by decile** (or uplift by percentile) plot.

You might be wondering – why are we talking about metrics and plots that are *cumulative* or *per decile*? The answer is related to the nature of the problem we're tackling: we never observe the true uplift (aka the true causal effect).

Let's take a look at *uplift by decile* and explain it step by step. It turns out that we can generalize it naturally to continuous outcomes. That's good news!

Uplift by decile

When we run an experiment, we can never observe all potential outcomes for a given unit at the same time. Each unit or subject can either receive treatment, T , or not receive it.

When we run a randomized experiment, we try to overcome this inherent limitation by computing group effects, assuming that the results will be generalizable (we could validly ask to what extent and under what conditions, but we will skip these questions here).

In other words, if we take a group of subjects and randomly assign them to experimental conditions, we hope that we can learn something about the general effect of the treatment.

Uplift by decile draws from the same source.

We divide the dataset into bins and assume we can learn something about the true effect within each bin. We leverage this information to assess the quality of our models.

How exactly do we do this?

1. First, we generate predictions from our models.
2. Next, we sort the predictions from the highest to the lowest predicted uplift.
3. We bin our predictions into deciles (in case of smaller datasets or when it's impossible to split the data into deciles, quantiles of smaller granularity can also be used, for example, quartiles).
4. We split the observations in our dataset into 10 bins according to the deciles we've just calculated.
5. Within each decile, we compute the average outcome for the units that were originally treated and the average outcome for the units that were originally in the control group.
6. Within each decile, we subtract the average outcome for untreated from the average outcome for treated.

These differences are our estimates of the *true uplift* within each decile.

7. We use a bar plot to visualize these estimates against the deciles, ordered from the top to the bottom decile (left to right).

Figure 10.11 presents an example of an uplift by decile plot:

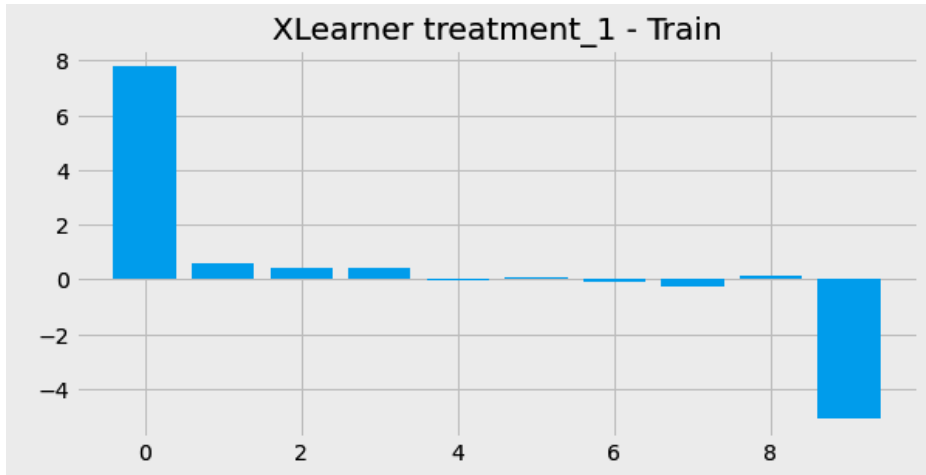


Figure 10.11 – An example of an uplift by decile plot

Let's take a closer look at *Figure 10.11*. The x axis represents deciles. The y axis represents our estimated average true uplift.

The *true meaning* of this plot comes from the fact that the deciles are *not* sorted by the *estimated true uplift* that we actually see in the plot but rather by our *predicted uplifts* (which are not visible in the plot).

How does this make sense?

We expect that the output of a good model will correlate with the true uplift – we'd like to see a high estimated true uplift in the top deciles (on the left) and a low estimated true uplift in the bottom deciles (on the right).

Figure 10.11 presents the results from a really good model as we can see that higher values are on the left and lower values are on the right.

In the case of a perfect model, the values on the y axis should *monotonically* go down from left to right. We see that in *Figure 10.11*, this is not entirely the case, with minor deviations on the fifth and eighth ticks (note that the x axis is 0-indexed), but overall, the pattern indicates a very good model.

Uplift by decile can be a good way to quickly visually assess the model, yet it doesn't provide us with the means to quantitatively compare various models.

We will use another tool for this purpose, but first, let's plot uplift by decile for all of our models. *Figure 10.12* presents the results:



Figure 10.12 – Uplift by decile plots

Each row in *Figure 10.12* corresponds to one model. The leftmost two columns present the results for treatment 1, and the rightmost two columns for treatment 2. Blue plots represent the evaluation on training data, while red ones the evaluation on the test data. Note that the y axes are not normalized. We did this on purpose to more clearly see the patterns between different models.

According to the criteria that we discussed earlier, we can say that most models perform very well on the training data. One exception is linear DML, with a less clear downward pattern. One reason for this might be the fact that the last stage in linear DML is... linear, which imposes a restriction on the model's expressivity.

When it comes to the test set, the performance of most models drops significantly. Most cells in the plot indicate poor performance. DR-Learner for treatment 1 is a strong exception, but the same model for treatment 2 gives almost a reversed pattern!

There might be a couple of reasons for this poor performance on the test set:

- First, our models might be overfitting to the training data. This can be related to the fact that there is not enough data for the models to build a generalizable representation.
- The second side of the same coin is that the architectures we used might be too complex for the task (note that we picked hyperparameters arbitrarily, without tuning the models at all).
- Finally, there might be some instability in the per-decile estimates of true uplift, yet the results on the train set suggest that this is likely not the main issue.

The fact that uplift per decile plots do not look favorable does not necessarily imply that our models are not useful.

Let's compute a metric that will help us assess whether the models can bring us some real value.

Expected response

The **expected response metric** was introduced by Yan Zhao and colleagues in their 2017 paper *Uplift Modeling with Multiple Treatments and General Response Types* (Zhao et al., 2017). The method works in multiple-treatment scenarios and with continuous outcomes, which is perfect for our case.

Although the metric is focused on the outcome rather than on uplift, it's a valid way to evaluate uplift models. The metric computes the expected average outcome of a model by combining information from all the treatments. That's very handy.

At the same time, the metric is also useful from a decision-making point of view as it gives us a good understanding of what average return on investment we can expect by employing a chosen model.

Intuitively, the metric works for uplift models because we use previously unseen data to check the expected return on investment for a given model. This information can be used to compare two or more models under unconfoundedness and obtain information on the expected performance of out-of-sample data.

Before we explain how the expected response metric works, let's get some context. Uplift models can be understood as **action recommendation systems**.

Imagine we have k different treatments. For each observation in the dataset, we can obtain k effect estimates – one for each treatment. We can pick the highest estimate for each observation and treat it as a predicted optimal treatment.

Now to the explanation.

The idea behind the *expected response* is simple. For each observation in the test set, we check whether the predicted treatment was the same as the actual treatment. If so, we divide the value of the outcome for this observation by the probability of the treatment and store it. Otherwise, we set the value to 0 and store it. Finally, we average over the stored values and the obtained score is our *expected response* for a given model.

This basically gives us a treatment-weighted average outcome that we expect to get if we administer treatments according to the model's recommendations.

I put the formula for the expected response metric in the notebook for this chapter. We'll skip it here, but feel free to explore the notebook and play with the implementation and the formula.

Now, let's see the results for the expected response (the full code is in the notebook):

```
Expected response on train:

SLearner: 2.004509358884219
TLearner: 2.499033323345889
XLearner: 2.3844783394170035
DRLearner: 2.357949913732327
LinearDML: 1.5596898804820036
CausalForestDML: 2.9583483277085936

-----
Expected response on test:

SLearner: 1.3498049695400602
TLearner: 1.2108070097957537
XLearner: 1.2901946274863276
DRLearner: 1.500974856751045
LinearDML: 1.3496321556492639
CausalForestDML: 1.3114730998145985
```

We see that the metric for all models significantly dropped between the train and test sets. The smallest difference can be observed for the linear DML model. This is congruent with what we observed in the uplift by decile plot – linear DML performed relatively poorly on the training data and slightly worse on the test data.

The best model on the test set according to the *expected response* is DR-Learner. This also translates – at least partially – to what we observed in the plot. DR-Learner had a pretty clear downward trend for treatment 1. Perhaps this good performance on treatment 1 allowed the model to compensate for the worse performance on treatment 2.

Other metrics for continuous outcomes with multiple treatments

Radcliffe proposed a version of Qini that is applicable to continuous outcome problems (Radcliffe, 2007), but it does not work with multiple treatments.

A multi-treatment generalization has been proposed by Gubela & Lessmann (2020). Check out their article for more details: <https://bit.ly/MultiTreatQini>.

Confidence intervals

We said before that one of the advantages of linear DML is its ability to provide us with valid confidence intervals. How can we obtain them in practice?

With EconML, it's very easy. We simply call the `.effect_interval()` method on the fitted estimator. The method returns a two-tuple of numpy arrays. The first array contains the lower bounds and the second the upper bounds of the confidence intervals.

Let's implement it!

```
models['LinearDML'].effect_interval(X=X_test, T0=0, T1=1)
```

The output is as expected:

```
(array([-1.41417908, -0.40901173, -1.50891344, ..., -0.27946152,
        -0.8185368 , -0.92689394]),
 array([0.88022207, 1.03706657, 3.34888932, ..., 2.47925003,
        1.75716167, 1.51788005]))
```

Based on the confidence intervals, we can decide whether there are observations that we'd like to exclude from targeting. Let's see how many intervals contain 0:

```
ints = np.stack(models['LinearDML'].effect_interval(
    X=X_test, T0=0, T1=1, alpha=.05)).T
# What % of effects contains zero?
(np.sign(ints[:, 0]) == np.sign(ints[:, 1])).sum() /
ints.shape[0]
```

This gives us the following:

```
0.100875
```

Out of all the test observations, confidence intervals for 10% of them contain 0. Removing these observations from our action recommendation set could perhaps further improve the model's performance.

To obtain confidence intervals for methods that do not support them natively, pass `inference='bootstrap'` to the model's `.fit()` method. Note that this will result in a significant increase in training time. The number of bootstrap samples can be adjusted by using the `BootstrapInference` object. For more details, check out <https://bit.ly/EconMLBootstrapDocs>.

Kevin’s challenge’s winning submission

At the beginning of this section, we said that the data we were using was part of a challenge. The submission that Kevin Hillstrom picked as the winning one came from Nicholas Radcliffe.

Does this name ring a bell for you?

If it sounds oddly familiar, it might be because we’ve already seen it (that is, the last name) in this chapter. Nicholas Radcliffe is not only the author of the winning submission in Kevin’s challenge but also the person who originally proposed the Qini coefficient (and the Qini curve).

If you’re interested in uplift modeling and marketing, you might find Nicholas’ submission interesting or even inspiring. The document describing Nicholas’ approach is available here: <https://bit.ly/NicksSolution>.

Before we close this section, I want us to think about a question that I received from one of the members of the **causalpython.io** community.

When should we use CATE estimators for experimental data?

Organizations – both commercial and non-commercial – usually decide to perform experiments in order to answer particular questions. A commercial organization might ask questions such as what impact a change in our funnel page design will have on our sales. A university research team might be interested in learning how a change in communication style from the municipality will impact citizens’ engagement in an important public initiative.

These questions can often be answered using randomized experiments and – in many cases – traditional statistical analyses.

A two-sample sample *t-Test* (Student, 1908) is a popular choice for analyzing the data from binary treatment experiments. For more complex designs, perhaps involving hierarchical structures, *mixed-effect models* can be used (e.g., Baayen et al., 2008; Barr, 2008). We also have a large variety of methods in between, including (but not limited to) regression analysis and – closely related to the latter – ANOVA.

All these methods can help us make valid conclusions from experimental data.

Experiments, treatment effects, and generalization

What *valid conclusions* from experiments really mean and what the generalizability guarantees are are beyond the scope of this book. If you’re interested in understanding the intricacies of the relationships between experimental results, the ATE, and transportability (the generalizability of results beyond the test (sub)population), this Twitter conversation involving Judea Pearl, Frank Harrell, and Stephen Senn, among others, might be interesting to you: <https://bit.ly/GeneralizationDiscussion>.

Machine learning methods come in handy when the feature space is large and when relationships and interactions in the data can take arbitrary functional forms. CATE methods in particular give us additional flexibility by enabling us to predict individualized responses to a (potentially high) number of treatments under high model complexity. These quasi-counterfactual capabilities go beyond what traditional statistical methods can offer us.

Uplift (CATE and HTE) modeling techniques have been proven to bring value in numerous industrial settings from marketing and banking to gaming and more.

The main price that we pay for the flexibility offered by CATE machine learning models is the difficulty of computing the sample size necessary to establish the desired statistical power. In *Chapter 9*, we discussed some ideas of how to overcome this limitation.

The main advantage of CATE machine learning models is that they give us a way to non- or semi-parametrically distinguish the units that can benefit from obtaining our treatment (*persuadables*) from those that will most likely not benefit from it (*sure things* and *lost causes*) and those who will likely not benefit but can additionally get hurt or hurt us (*do not disturbs*; check *Chapter 9* and *Table 9.5* for a refresher).

More ways to determine the sample size for complex models

When it comes to estimating statistical power for complex models, there's a smart trick that I recently learned from Frank Harrell (<https://bit.ly/FrankHarrell>). If you can afford a pilot study or you have some historical data that represents a problem similar to the one that you're interested in, you can find a subgroup in your data that is as homogenous as possible. You can estimate the sample size for this group using some of the traditional statistical power tools. Finally, scale your overall sample size so that this subgroup is properly powered relative to the entire sample (Harrell, 2023).

Model selection – a simplified guide

Before closing this section, I want to share a simple table with you, where we summarize key information about the models we discussed in this chapter. I hope that this will help you get the most out of this chapter.

Table 10.3 summarizes important aspects of various EconML estimators that can help you guide your implementation decisions:

Model name	Treatment type	Confidence intervals	Linear treatment assumed	Multiple outcomes	Training speed (relative to S-Learner)
S-Learner	Categorical (can be adapted to continuous)	Only by bootstrapping	No	Yes	1x
T-Learner	Categorical	Only by bootstrapping	No	Yes	2x
X-Learner	Categorical	Only by bootstrapping	No	Yes	5x
DR-Learner	Categorical	Only by bootstrapping	No	No	13x
LinearDML	Categorical, continuous	Natively	Yes	Yes	27x
CausalForestDML	Categorical, continuous	Bag-of-little-bootstraps or by (conventional) bootstrapping	Yes	Yes	39x

Table 10.3 – Comparison of selected EconML estimators

For an even more comprehensive comparison between EconML estimators, check out this documentation page: <https://bit.ly/EconMLComparison>.

For more performance comparisons between various machine learning methods for CATE (meta-learners, DR, and more) on experimental data, check Jacob (2021).

Now, it's time to conclude this section.

We started this section with a brief discussion on using CATE models with experimental data and the EconML workflow tailored to experimental scenarios.

After that, we introduced the Hillstrom dataset and tested whether our data is unconfounded under observed variables. We fitted six different models and compared their performance from a computational cost point of view.

We introduced popular evaluation tools for uplift models: the Qini coefficient, the AUUC, and uplift by decile. We used the latter and the generalized *expected response metric* to compare our models' performance.

Finally, we showed how to obtain confidence intervals from EconML estimators, discussed when using CATE machine learning models with experimental data can be beneficial, and shared a comprehensive causal estimator comparison table.

In the next short section, we'll discuss the idea of using machine learning for counterfactual explanations.

Extra – counterfactual explanations

Imagine that you apply for a loan from your bank. You prepared well – you checked your credit score and other variables that could affect the bank's decision. You're pretty sure that your application will be approved.

On Wednesday morning, you see an email from your bank in your inbox. You're extremely excited! You open the message, already celebrating and ready to welcome the success!

There's a surprise in the email.

Your loan application has been rejected.

You call the bank. You ask questions. You want to understand why. At the end of the day, its decision impacts some of your most important plans!

The only response you get from the customer service representative is that *you did not meet the criteria*. "Which criteria?" you ask. You don't get a satisfying answer.

You'd like to make sure that you meet the criteria the next time you re-apply, yet it seems that no one can tell you *how* to improve!

Bad faith or tech that does not know?

Does the lack of answers come from bank employees' unwillingness to help?

Not necessarily. Some organizations might not be technically ready to answer questions like ours. How could we help them change this?

Let's recall the idea behind S-Learner.

First, we train a single model on our data. Then, we generate predictions for different values of the treatment variable and perform a simple subtraction. This idea is very flexible. What if we extended it to other variables that we did not treat as the treatment before?

Theoretically (depending on the setting), this could mess up the causal character of the model.

Do we care?

If our goal is to answer the question *What should be done?* in order to *change the output of the model*, we actually do not.

Why?

We're performing an *intervention on the model* and we're interested in the *model's behavior*, not in how accurately it represents an underlying process.

Interventions such as this provide us with valid results *given our goal*.

In a sense, this approach is even simpler than S-Learner as we're only interested in finding minimal changes to the data that would result in the expected change of the outcome, so we don't even need to calculate the causal effect. There are some challenges that come with this approach, though.

What if we need to change more than one feature in order to influence the outcome because there's an interaction between these features? What if there are many ways to change the outcome and some are much easier, but we've only found the hardest one?

To address this and other issues, Microsoft released an open source package called **Diverse Counterfactual Explanations** (DiCE; Mothilal et al., 2020). The basic intuition behind DiCE is that it searches for a set of changes in the input features that lead to the outcome change, at the same time maximizing the proximity to the original values and diversity (finding many different solutions to the same problem to let us choose the best one). If you want to learn more about DiCE, check out the introductory blog post by Amit Sharma (<https://bit.ly/DiCEIntro>) and DiCE's GitHub repository (<https://bit.ly/DiCERepo>).

Note that depending on the industry sector and geopolitical region, usage of methods such as the ones we discussed previously might be difficult due to particular regulatory requirements. DiCE offers a smart set of tools that can help when sensitive data is at stake.

In cases where complex machine learning models might be difficult to apply due to interpretability requirements, logic similar to that we discussed at the beginning of this section can be implemented with simple linear models.

In this short extra section, we discussed basic ideas behind counterfactual model explanations and introduced DiCE – an open source Python library that helps analysts perform counterfactual inferences easily in more complex cases.

Wrapping it up

Congratulations! You just reached the end of *Chapter 10*.

In this chapter, we introduced four new causal estimators: DR-Learner, TMLE, DML, and Causal Forest. We used two of them on our synthetic earnings dataset, comparing their performance to the meta-learners from *Chapter 9*.

After that, we learned about the differences in workflows between observational and experimental data and fit six different models to the Hillstrom dataset. We discussed popular metrics used to evaluate uplift models and learned how to use confidence intervals for EconML estimators. We discussed